

JAPANESE EARTHQUAKE WARNING SYSTEM

1. Descrierea Generală a Proiectului

- **Ce este J-EWS:** Un sistem distribuit local de avertizare timpurie în caz de seism (Japan Earthquake Warning System).
- **Componente:**
 - `ews-common.h`: Header-ul cu protocoalele și structurile de date.
 - `ews-sensor.c`: Procesul client care simulează citirile seismice și trimite semnale de viață ale senzorilor.
 - `ews-daemon.c`: Creierul sistemului care rulează în background ca serviciu.
 - `ews-dashboard.c`: Interfața grafică în terminal pentru administrator.

2. Arhitectura IPC și Concepte de Sisteme de Operare Utilizate

- **Multiplexare IPC (Message Queues):** Comunicarea se face printr-o singură coadă de mesaje System V, separată prin câmpul `mtype` (`mtype=1` pentru date seismice, `mtype=2` pentru statistici/avarii dashboard, `mtype=3` pentru heartbeat).
- **Proces Daemon:** Serverul este izolat complet de terminalul de control prin tehnica dublului `fork()`, `setsid()` și ignorarea semnalului `SIGHUP`.
- **Grup de Thread-uri (Thread Pool) și Excludere Mutuală (Mutex):** Daemonul folosește thread-uri paralele pentru procesarea mesajelor și un thread pentru colectarea pulsului. Variabilele globale sunt protejate de un lacăt `pthread_mutex_t` pentru a preveni problemele de tip *Data Race*.
- **Mecanism Watchdog (Heartbeat):** Detecția proactivă a căderii senzorilor prin monitorizarea timpului scurs de la ultimul puls (`mtype=3`), cu un timeout de 15 secunde gestionat în bucla principală a Daemonului.
- **Semnale POSIX Asincrone:** Utilizarea funcției `kill ()` pentru a trimite semnalul personalizat `SIGUSR1` de la Daemon la Dashboard în caz de cutremur critic, interceptat prin `sigaction`.

3. Ghid de Compilare și Rulare

- `gcc -Wall -o daemon ews-daemon.c -lpthread`
- `gcc -Wall -o dashboard ews-dashboard.c`

- `gcc -Wall -o sensor_ews-sensor.c -lpthread`

Ghid de prezentare

PASUL 1: Introducere și Arhitectura de Rețea

- **Obiectiv:** Explicarea conceptului și a modului în care componentele comunică prin Kernel.
- **Idei principale de susținut:**
 - Definirea J-EWS ca sistem local distribuit, axat pe *fault-tolerance* (rezistență la erori) și procesare concurentă.
 - Prezentarea celor 3 componente: Producătorul (ews-sensor), Creierul (ews-daemon) și Consumatorul/Vizualizatorul (ews-dashboard).

PASUL 2: Demonstrația Practică a Background-ului

- **Obiectiv:** Demonstrarea izolării proceselor și pornirea curată a sistemului.
- **Acțiuni în terminal și explicații în timp real:**
 - **Rularea Daemonului (./daemon):** Se arată cum terminalul cedează instant controlul. Se explică mecanismul de *daemonizare UNIX*: dublu fork(), apelul setsid() pentru sesiune nouă, ignorarea SIGHUP (rezistență la închiderea terminalului) și redirectarea descriptorilor prin dup2 în ews.log.
 - **Rularea Dashboard-ului (./dashboard):** Se explică cum procesul își înregistrează PID-ul în dashboard.pid și folosește sigaction cu flag-ul SA_RESTART pentru a intercepta semnalul asincron SIGUSR1.
 - **Rularea Senzorilor (./senzor 1 & ./senzor 2 &):** Se explică conceptul de *Background Job Control* (caracterul &) și faptul că senzorul rulează multi-threaded (un fir pentru cutremure aleatorii, un fir tip ceasornic la 5s pentru puls).

PASUL 3: Sincronizarea și Logica Watchdog

- **Obiectiv:** Demonstrarea modului în care sistemul gestionează concurența și detectează automat avariile (Auto-Healing).
- **Acțiuni în terminal și explicații în timp real:**
 - **Explicarea Zonei Critice:** În timp ce datele curg, se precizează că pool-ul de thread-uri Worker din Daemon accesează variabilele globale (nivel_pericol_global, mesaje_procesate) în deplină siguranță, fiind protejat

de un lacăt pthread_mutex_t pentru eliminarea competiției de memorie (*Data Race*).

- **Simularea Avariei (killall senzor):** Se ucid procesele senzorilor din terminal.
- **Demonstrația Watchdog:** Se explică cum bucla main din Daemon acționează ca un *câine de pază*. La fiecare 2 secunde, inspectează vectorul de timp global. Dacă diferența time(NULL) - ultima_bataie[i] depășește **15 secunde**, se declanșează alerta.
- **Afișarea pe ecran:** Se urmărește Dashboard-ul cum trece automat în starea AVARIE / SENZOR PICAT.

PASUL 4: Concluzii și Validare Academică

- **Concluzii de reținut:**

- S-au atins pilonii de bază ai disciplinei: managementul proceselor (Daemoni), concurență (Thread-uri), excludere mutuală (Mutex), semnale POSIX customizate (SIGUSR1 prin kill()) și magistrale IPC (Coadă de mesaje).
- Sistemul este complet autonom, asincron și capabil să își mențină starea și logurile independent de utilizator.